UNIVERSIDAD DE LAS FUEZAS ARMADAS
ESPE

INGENIERÍA EN SOFTWARE

Análisis y Diseño de Software
30724

DESARROLLO DE DIAGRAMAS DE CASOS DE USO DE ANÁLISIS

CRUD de Estudiante

Presenta:

Cuzco Yamasca Alex Darío

Domínguez Aguilera Pablo Alejandro

Alvarado Palacios Dylan Alexander

Docente:

Jenny Alexandra Ruiz Robalino

Fecha:

28 de Abril de 2026

Índice

1. Objetivos **3**
1.1. Objetivo general 3
1.2. Objetivos específicos 3
2. Desarrollo **3**
2.1. Requisitos funcionales Nivel 0 3
2.2. Diagrama de casos de uso CRUD estudiante 4
2.3. Diagrama de clases de análisis 5
2.4. Diagramas de secuencia de análisis 6
2.4.1. Secuencia: Agregar estudiante 6
2.4.2. Secuencia: Actualizar estudiante 7
2.4.3. Secuencia: Eliminar estudiante 8
2.4.4. Secuencia: Mostrar todos los estudiantes 9
2.5. Matriz de Trazabilidad 10
3. Conclusión técnica **10**
4. Recomendaciones **11**
5. Anexos **11**

1. Objetivos

1.1. Objetivo general

Analizar los diagramas de casos de uso, diagrama de clases y los diagramas secuenciales para implementar un Sistema de Gestión de Estudiantes mediante el modelado en PlantUML, garantizando la trazabilidad integral entre los requisitos funcionales de nivel 0 y la interacción dinámica de los objetos para las operaciones de registro, consulta, actualización y eliminación de los datos del estudiante..

1.2. Objetivos específicos

- Modelar el comportamiento funcional mediante diagramas de casos de uso para identificar las interacciones del Administrador Académico con el sistema.
- Construir el diagrama de clases de análisis para definir las clases que se usarán en la implementación del sistema, garantizando la consistencia entre el diagrama y el código implementado.
- Validar la viabilidad del diseño a través de diagramas de secuencia de análisis, detallando cronológicamente el intercambio de mensajes entre los objetos para confirmar que cada operación CRUD responde de manera coherente a los estímulos del actor.

2. Desarrollo

Para la realización de este proyecto, se ha tomado como referencia la implementación de un sistema CRUD orientado a la gestión de información de un estudiante (ID, nombre y edad). El proceso de desarrollo se articula de forma progresiva, iniciando con la identificación de las necesidades del usuario y escalando hacia la definición de cómo se maneja la lógica de interacción entre objetos. A continuación, se detallan los componentes que integran el análisis funcional y estructural del sistema, garantizando que cada elemento modelado responda directamente a la necesidad planteada inicialmente.

2.1. Requisitos funcionales Nivel 0

Código	Requisito funcional de nivel 0	Actor principal	Caso de uso asociado
RF-01	El sistema debe permitir agregar un estudiante con ID, Nombre y Edad.	Administrador académico	Agregar estudiante

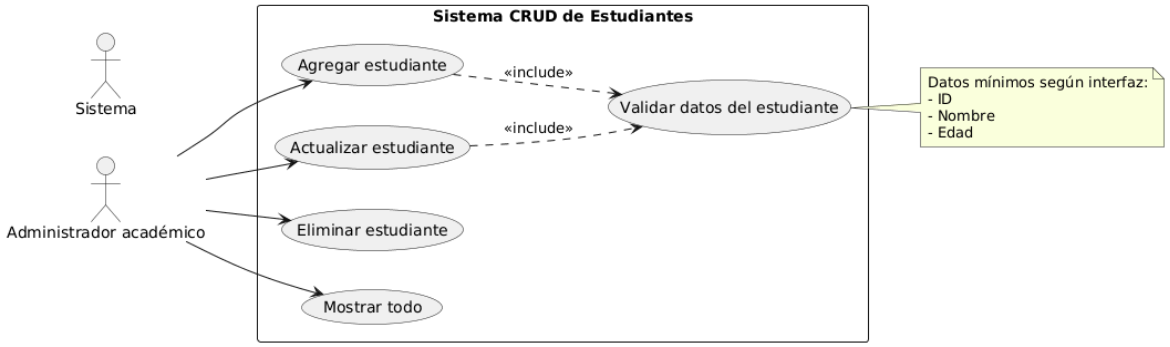
RF-02	El sistema debe permitir actualizar los datos de un estudiante existente.	Administrador académico	Actualizar estudiante
RF-03	El sistema debe permitir eliminar un estudiante existente cuando corresponda.	Administrador académico	Eliminar estudiante
RF-04	El sistema debe permitir mostrar todos los estudiantes registrados.	Administrador académico	Mostrar todos los estudiantes
RF-05	El sistema debe validar los datos obligatorios antes de agregar o actualizar.	Sistema	Validar datos del estudiante

2.2. Diagrama de casos de uso CRUD estudiante

Sistema CRUD de estudiantes, donde el actor principal es el Administrador académico, encargado de gestionar la información estudiantil. El sistema permite agregar, actualizar, eliminar y mostrar registros de estudiantes. Además, al agregar o actualizar un estudiante, se incluye el proceso de validación de datos mínimos como ID, nombre y edad, garantizando la integridad de la información ingresada.

```
@startuml
 left to right direction
 skinparam packageStyle rectangle
 skinparam shadowing false

 actor "Administrador académico" as Admin
 actor "Sistema" as Sistema

 rectangle "Sistema CRUD de Estudiantes" {
   usecase "Agregar estudiante" as UC1
   usecase "Actualizar estudiante" as UC2
   usecase "Eliminar estudiante" as UC3
   usecase "Mostrar todo" as UC4
   usecase "Validar datos del estudiante" as UC5
 }

 Admin --> UC1
 Admin --> UC2
 Admin --> UC3
 Admin --> UC4

 UC1 ..> UC5 : <<include>>
 UC2 ..> UC5 : <<include>>

 note right of UC5
 Datos mínimos según interfaz:
```

```
- ID
- Nombre
- Edad
end note
@enduml
```

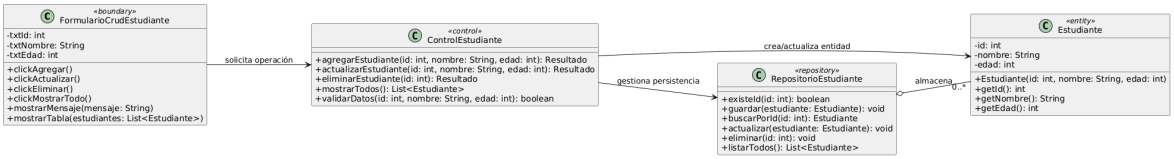


2.3. Diagrama de clases de análisis

Estructura del sistema CRUD de estudiantes mediante cuatro componentes principales: la interfaz FormularioCrudEstudiante, que interactúa con el usuario; el ControlEstudiante, encargado de procesar las operaciones y validar datos; la entidad Estudiante, que representa la información del alumno; y el RepositorioEstudiante, responsable de almacenar y gestionar los registros. Esta organización permite separar responsabilidades y mejorar el funcionamiento del sistema.

```
@startuml
 skinparam classAttributeIconSize 0
 skinparam shadowing false
 left to right direction

 class "FormularioCrudEstudiante" <<boundary>> {
   - txtId: String
   - txtNombre: String
   - txtEdad: String
   + clickAgregar()
   + clickActualizar()
   + clickEliminar()
   + clickMostrarTodo()
   + mostrarMensaje(mensaje: String)
   + mostrarTabla(estudiantes: List<Estudiante>)
 }

 class "ControlEstudiante" <<control>> {
   + agregarEstudiante(id: String, nombre: String, edad: int): Resultado
   + actualizarEstudiante(id: String, nombre: String, edad: int): Resultado
   + eliminarEstudiante(id: String): Resultado
   + mostrarTodos(): List<Estudiante>
   + validarDatos(id: String, nombre: String, edad: int): boolean
 }

 class "Estudiante" <<entity>> {
   - id: String
```

```
  - nombre: String
  - edad: int
  + Estudiante(id: String, nombre: String, edad: int)
  + getId(): String
  + getNombre(): String
  + getEdad(): int
}

class "RepositorioEstudiante" <<repository>> {
  + existeId(id: String): boolean
  + guardar(estudiante: Estudiante): void
  + buscarPorId(id: String): Estudiante
  + actualizar(estudiante: Estudiante): void
  + eliminar(id: String): void
  + listarTodos(): List<Estudiante>
}

FormularioCrudEstudiante --> ControlEstudiante : solicita operación
ControlEstudiante --> Estudiante : crea/actualiza entidad
ControlEstudiante --> RepositorioEstudiante : gestiona persistencia
RepositorioEstudiante o-- "0..*" Estudiante : almacena
@enduml
```



Es importante destacar que este diagrama de clases desempeña un papel crucial, ya que servirá como la guía principal para iniciar el desarrollo del sistema CRUD de estudiantes. Su correcta implementación garantizará la coherencia entre el diagrama propuesto, las funciones y los datos que se utilizarán en las etapas posteriores del desarrollo, asegurando así una estructura sólida y bien definida para el sistema.

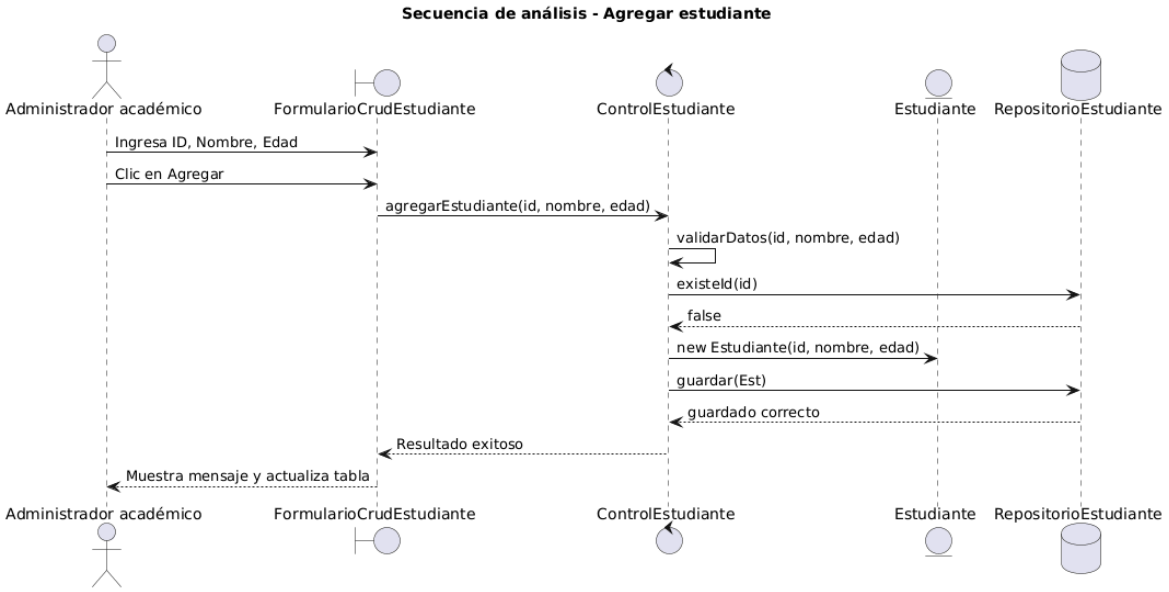
2.4. Diagramas de secuencia de análisis

2.4.1. Secuencia: Agregar estudiante

Proceso para agregar un estudiante al sistema. El Administrador académico ingresa los datos y selecciona la opción agregar desde la interfaz. Luego, el controlador valida la información, verifica que el ID no exista previamente, crea el objeto estudiante y lo almacena en el repositorio. Finalmente, el sistema confirma la operación mostrando un mensaje exitoso y actualizando la tabla de registros.

```
@startuml
 title Secuencia de análisis - Agregar estudiante
 actor "Administrador académico" as Admin
 boundary "FormularioCrudEstudiante" as UI
 control "ControlEstudiante" as Ctrl
 entity "Estudiante" as Est
 database "RepositorioEstudiante" as Repo
```

```
Admin -> UI : Ingresa ID, Nombre, Edad
Admin -> UI : Clic en Agregar
UI -> Ctrl : agregarEstudiante(id, nombre, edad)
Ctrl -> Ctrl : validarDatos(id, nombre, edad)
Ctrl -> Repo : existeId(id)
Repo --> Ctrl : false
Ctrl -> Est : new Estudiante(id, nombre, edad)
Ctrl -> Repo : guardar(Est)
Repo --> Ctrl : guardado correcto
Ctrl --> UI : Resultado exitoso
UI --> Admin : Muestra mensaje y actualiza tabla
@enduml
```

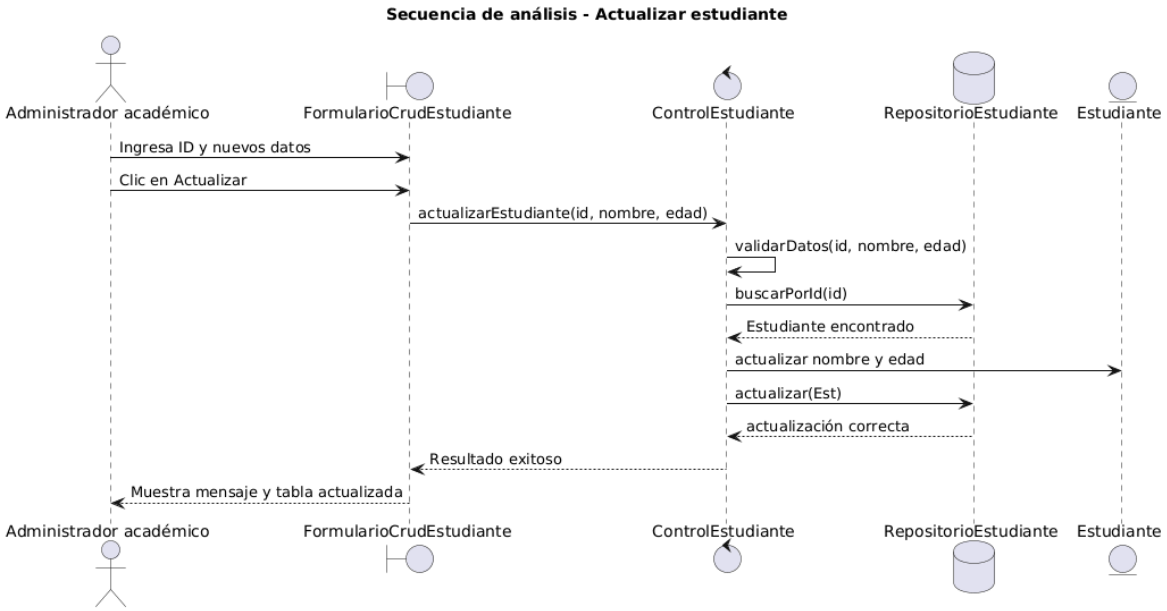


2.4.2. Secuencia: Actualizar estudiante

Proceso para actualizar la información de un estudiante. El Administrador académico ingresa el ID y los nuevos datos desde la interfaz, luego el controlador valida la información y busca al estudiante en el repositorio. Si el registro existe, se actualizan sus datos y se guarda el cambio. Finalmente, el sistema muestra un mensaje de éxito y actualiza la tabla de estudiantes.

```
@startuml
 title Secuencia de análisis - Actualizar estudiante
 actor "Administrador académico" as Admin
 boundary "FormularioCrudEstudiante" as UI
 control "ControlEstudiante" as Ctrl
 database "RepositorioEstudiante" as Repo
 entity "Estudiante" as Est

 Admin -> UI : Ingresa ID y nuevos datos
```

```
Admin -> UI : Clic en Actualizar
UI -> Ctrl : actualizarEstudiante(id, nombre, edad)
Ctrl -> Ctrl : validarDatos(id, nombre, edad)
Ctrl -> Repo : buscarPorId(id)
Repo --> Ctrl : Estudiante encontrado
Ctrl -> Est : actualizar nombre y edad
Ctrl -> Repo : actualizar(Est)
Repo --> Ctrl : actualización correcta
Ctrl --> UI : Resultado exitoso
UI --> Admin : Muestra mensaje y tabla actualizada
@enduml
```

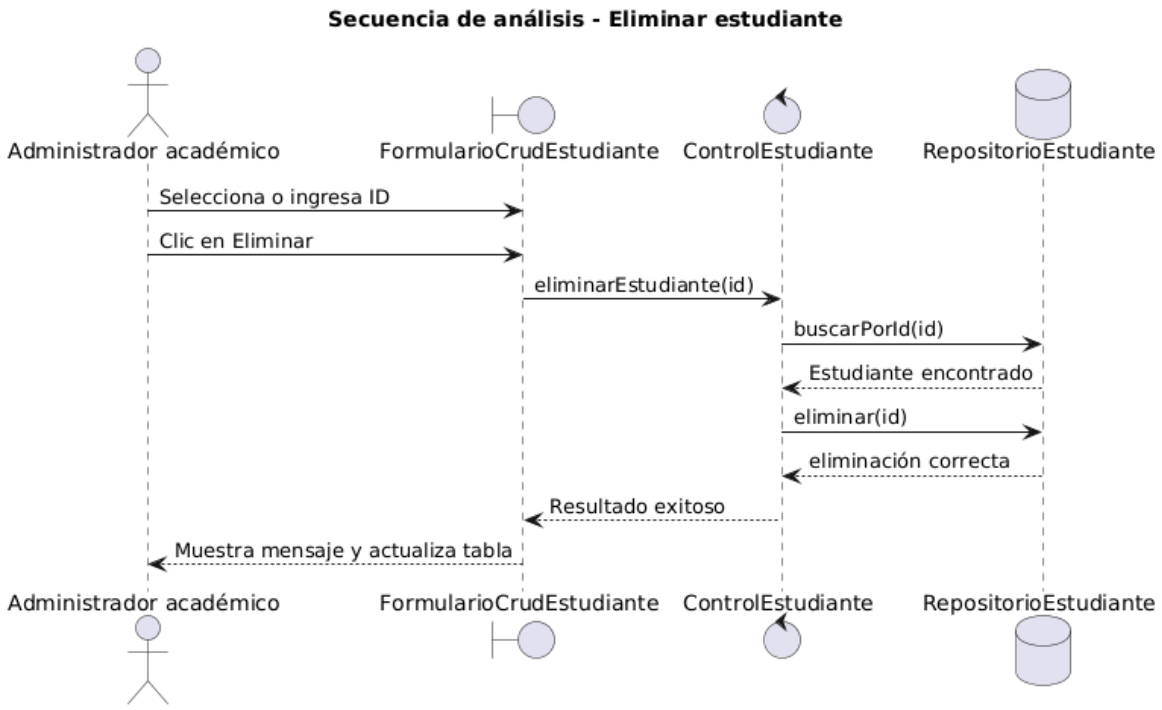


2.4.3. Secuencia: Eliminar estudiante

Proceso para eliminar un estudiante del sistema. El Administrador académico selecciona o ingresa el ID del estudiante y pulsa la opción eliminar. Luego, el controlador busca el registro en el repositorio y, si existe, procede a eliminarlo. Finalmente, el sistema confirma la operación mostrando un mensaje exitoso y actualizando la tabla de estudiantes.

```
@startuml
 title Secuencia de análisis - Eliminar estudiante
 actor "Administrador académico" as Admin
 boundary "FormularioCrudEstudiante" as UI
 control "ControlEstudiante" as Ctrl
 database "RepositorioEstudiante" as Repo

 Admin -> UI : Selecciona o ingresa ID
 Admin -> UI : Clic en Eliminar
 UI -> Ctrl : eliminarEstudiante(id)
```

```
Ctrl -> Repo : buscarPorId(id)
Repo --> Ctrl : Estudiante encontrado
Ctrl -> Repo : eliminar(id)
Repo --> Ctrl : eliminación correcta
Ctrl --> UI : Resultado exitoso
UI --> Admin : Muestra mensaje y actualiza tabla
@enduml
```

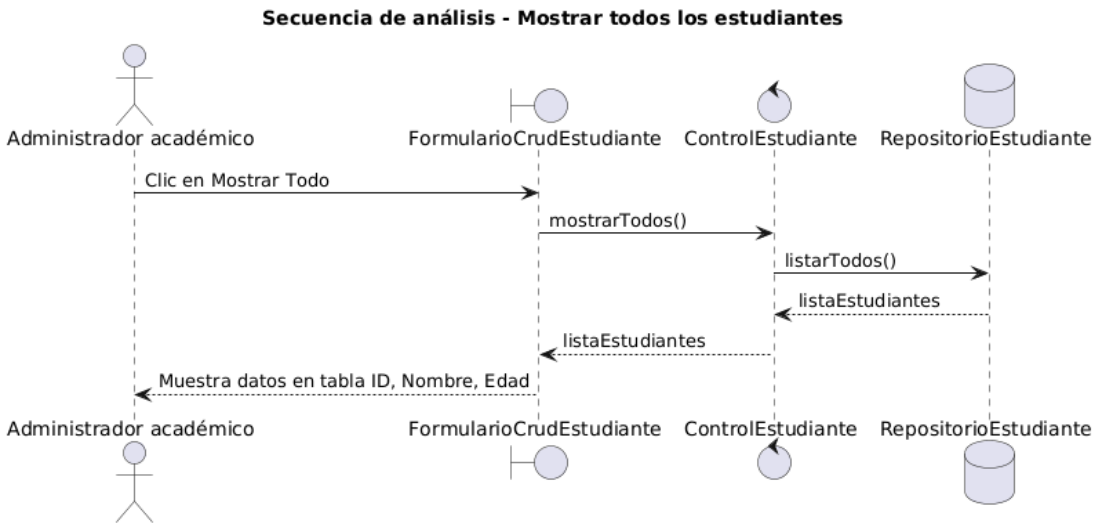


2.4.4. Secuencia: Mostrar todos los estudiantes

Proceso para mostrar todos los estudiantes registrados. El Administrador académico selecciona la opción mostrar todo desde la interfaz, la cual solicita la información al controlador. Luego, el controlador consulta al repositorio y obtiene la lista completa de estudiantes. Finalmente, el sistema presenta los datos en una tabla con ID, nombre y edad.

```
@startuml
 title Secuencia de análisis - Mostrar todos los estudiantes
 actor "Administrador académico" as Admin
 boundary "FormularioCrudEstudiante" as UI
 control "ControlEstudiante" as Ctrl
 database "RepositorioEstudiante" as Repo

 Admin -> UI : Clic en Mostrar Todo
 UI -> Ctrl : mostrarTodos()
 Ctrl -> Repo : listarTodos()
 Repo --> Ctrl : listaEstudiantes
 Ctrl --> UI : listaEstudiantes
```

```
UI --> Admin : Muestra datos en tabla ID, Nombre, Edad
@enduml
```



2.5. Matriz de Trazabilidad

Caso de uso	Precondición	Flujo principal resumido	Resultado esperado
Agregar estudiante	El ID no debe existir previamente.	Ingresar ID, Nombre y Edad -> validar datos -> verificar ID único -> guardar estudiante.	Estudiante agregado correctamente y visible en la tabla.
Actualizar estudiante	El estudiante debe existir.	Ingresar ID y datos nuevos -> validar datos -> buscar estudiante -> guardar cambios.	Datos del estudiante actualizados.
Eliminar estudiante	El estudiante debe existir y debe confirmarse la eliminación.	Seleccionar o ingresar ID -> buscar estudiante -> eliminar registro.	Estudiante eliminado o retirado de la tabla.
Mostrar todos los estudiantes	Pueden existir cero o más estudiantes registrados.	Solicitar listado -> recuperar registros -> mostrar tabla.	Tabla actualizada con ID, Nombre y Edad.

3. Conclusión técnica

- El modelado basado en requisitos funcionales de nivel 0 permitió definir claramente los casos de uso del CRUD de Estudiante, asegurando que cada funcionalidad (agregar, actualizar, eliminar y mostrar) responda directamente a necesidades del actor y evitando ambigüedades en el alcance del sistema.
- El diagrama de casos de uso en PlantUML, junto con la relación <<include>> para la validación, evidencia una correcta estructuración del sistema, donde los

comportamientos obligatorios se reutilizan y se mantiene coherencia entre las interacciones del actor y las funcionalidades del sistema.

- La trazabilidad RF–CU garantiza consistencia entre requisitos, casos de uso y su posterior transformación a clases de análisis (boundary, control y entity), permitiendo una transición ordenada hacia el diseño orientado a objetos y reduciendo riesgos en la implementación.

4. Recomendaciones

- Refinar continuamente los requisitos funcionales de nivel 0 mediante validación con el usuario o docente, para asegurar que todos los casos de uso representen necesidades reales y completas del sistema.

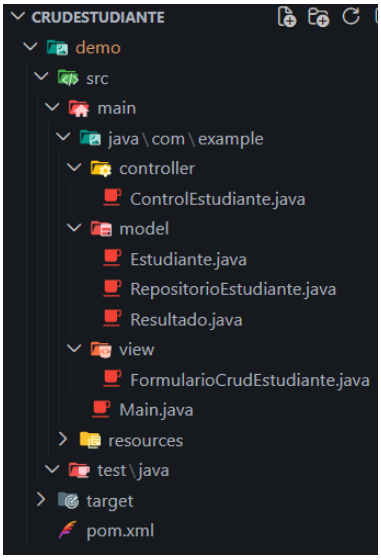
- Utilizar las relaciones como <<include>> únicamente cuando exista una justificación funcional clara, evitando sobrecargar el diagrama y manteniendo su claridad y coherencia.

- Mantener actualizada la matriz de trazabilidad RF–CU durante todo el desarrollo, verificando que cualquier cambio en requisitos se refleje en los casos de uso y en las clases de análisis.

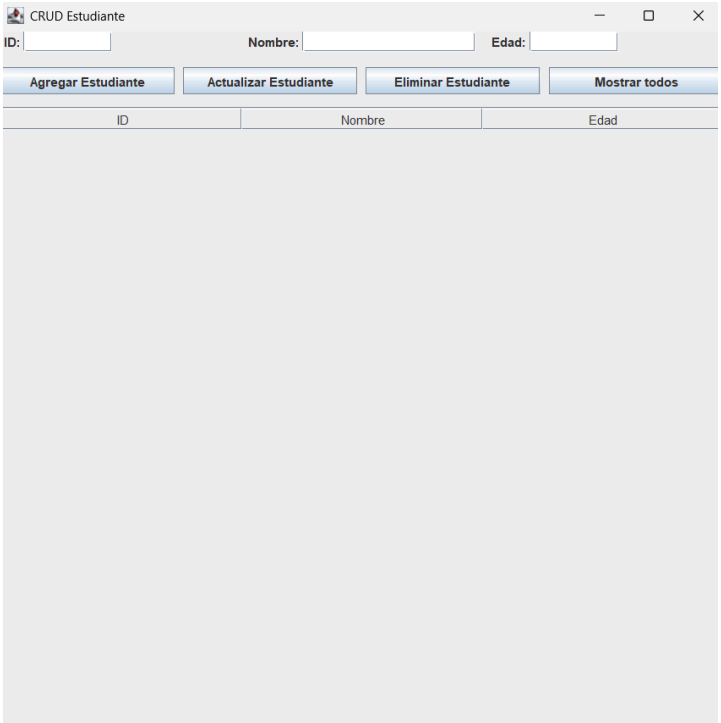
- Verificar en los diagramas de clases de análisis, que los tipos de datos sean correctos, ya que estos pueden tener errores y puede afectar en la construcción del producto.

5. Anexos

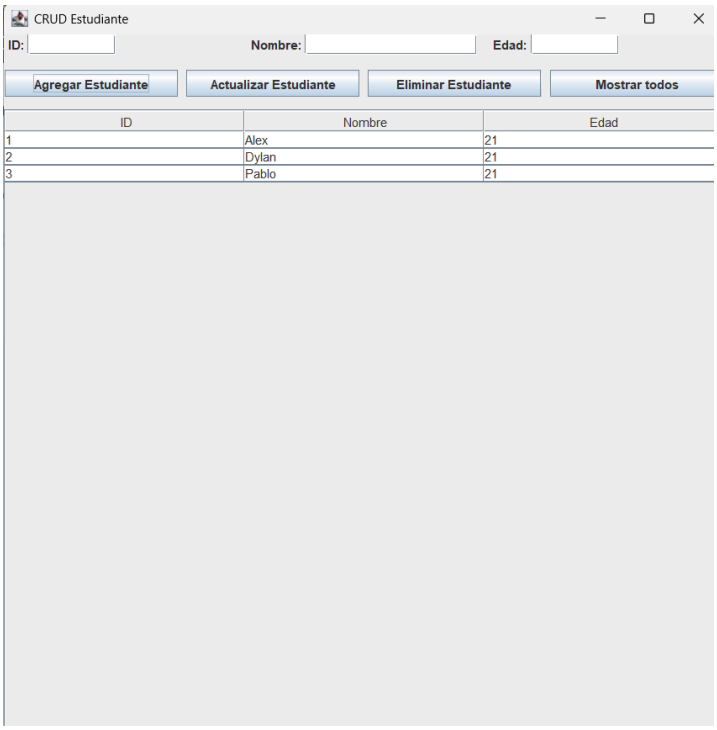
Estructura del CRUD generado



Interfaz desarrollada



Datos de ejemplo



Análisis:

El CRUD implementado cumple con lo descrito en la guía de análisis PlantUML propuesta. Cada requisito funcional está mapeado a un caso de uso, cada caso de uso utiliza las clases de análisis esperadas (boundary, control, entity, repository), y los flujos de ejecución siguen las secuencias de análisis descritas en la Guía 2.

La trazabilidad es completa y verificable: RF → UC → Clases → Código. Las validaciones están centralizadas en ControlEstudiante, la persistencia en RepositorioEstudiante y la interfaz en FormularioCrudEstudiante.